

NYC Mobility Explorer - Technical Report

1. Problem Framing and Dataset Analysis

Dataset Description

The NYC Yellow Taxi Trip dataset consists of NYC Taxi and Limousine Commission (TLC) data for trips taken throughout the City of New York starting in January 2019. This data contained information about each taxi trip, including the pick-up and drop-off datetime, zone where pick-up took place / where drop-off took place, ID of the zone, number of passengers, distance travelled, fare splits, payment method, surcharges etc.

Data Challenges

Missing Fields:

For a number of records, there were values of NULL in the PULocationID and DOLocationID fields.

Approximately 2% of records did not contain any payment type info.

For a number of trips, -Rate code ID was null.

Outliers and Anomalies:

Issue	Threshold	Count Removed
Dropoff before pickup	dropoff_datetime <= pickup_datetime	~150,000
Invalid distance	trip_distance <= 0	~120,000
Excessive distance	trip_distance > 150 miles	~50,000
Invalid fare	fare_amount <= 0 or > \$500	~110,000
Invalid passenger count	passenger_count <= 0 or > 6	~20,000
Wrong year	pickup year != 2019	~7,770

Temporal Anomalies:

- There were some trips that were not realistic in terms of the times taken (e.g. 3 hours for 1 mile).
- Congestion surcharge was NOT available for any of the trips during January 2019 (feature not active yet).

Data Cleaning Assumptions

1. Fare Amount: If fare_amount was less than or equal to 0, then the trip was not included – this may be due to incorrect data entry or the trip was a zero fare trip (and therefore of no value for analysis purposes). Trips costing more than \$500 were removed due to them being outliers and potentially not yellow taxi trips, but rather limousine or luxury service rides.

2. Trip Distance: Zero or negative distances were removed as data errors. Trips exceeding 150 miles were removed as outliers representing intercity travel

3. Passenger Count: Values between 1-6 are considered valid. Zero represents data error; >6 represents unusual circumstances.

4. Temporal Logic: Dropoff is needed to happen after pickup. These records were removed if they did not meet this criteria.

5. Year Filter: Only data from year 2019 was retained for consistency of the data set.

Unexpected Observation

We calculated that the average distance travelled in Staten Island is significantly higher than Manhattan (6.2 miles vs. 3.1 miles) while the average fare in Manhattan is \$12.50, compared to Staten Island's \$8.20. That could mean Manhattan is denser, with more frequent short trips, and potentially could have higher base fares, or congestion pricing impacts. This influenced our design, which was to display scaled per-borough (not global) map this demonstrated patterns that were not visible otherwise.

2. System Architecture and Design Decisions.

Stack Justification

Component	Choice	Rationale
Backend Framework	Flask	Lightweight, simple to set up, perfect for single-file API
Database	SQLite	No separate server needed, handles 7.49M rows efficiently with WAL mode
Frontend	Vanilla JS	No framework overhead, direct DOM manipulation, faster load times
Charts	Chart.js	Simple API, responsive, good browser support
Mapping	Leaflet.js	Lightweight, GeoJSON support, free tile layers
Caching	SessionStorage	Simple, no server-side cache setup needed

Schema Design

Normalization Level: 3NF (Third Normal Form)

Tables:

zones (Dimension Table):

Column	Type	Description
zone_id	INTEGER PK	TLC location ID
borough	TEXT	NYC borough
zone_name	TEXT	Human-readable name
service_zone	TEXT	Yellow/Green zone

trips (Fact Table):

Column	Type	Description
trip_id	INTEGER PK	Auto-increment
vendor_id, pickup_datetime, dropoff_datetime	various	Raw trip data
passenger_count, trip_distance	INTEGER/REAL	Trip metrics
pu_location_id, do_location_id	INTEGER	FK to zones
fare_amount, tip_amount, total_amount	REAL	Financial metrics
trip_speed_mph, time_of_day, fare_per_mile, is_weekend	various	Derived features

summary_cache (Aggregate Table):

Column	Type	Description
--------	------	-------------

total_trips, total_revenue, avg_fare, avg_distance, avg_tip_pct, peak_hour	various	Pre-calculated KPI data
borough-specific metrics	various	Per-borough pre-calculated data

Trade-offs

Decision	Trade-off	Rationale
SQLite vs PostgreSQL	PostgreSQL has better concurrency, SQLite is simpler	Small team, educational project, SQLite sufficient
SessionStorage vs Redis	Redis is faster, SessionStorage is simpler	No need for server-side caching
50 trips per page vs 200	Smaller pages = more API calls, larger pages = slower loads	Balanced at 50 for UX
Per-borough vs global color scaling	Per-borough shows patterns, global shows comparisons	Per-borough reveals hidden patterns in each borough
GeoJSON in frontend vs database	Frontend loads once, database would be faster	Simpler implementation, map renders locally

3. Algorithmic Logic and Data Structures

Problem Definition

The number of trips in the NYC taxi data set ranges from 265 zones. The application needs to display the top K pickup zones on the dashboard. Both `ORDER BY` and `LIMIT` in SQL would suffice, but our assignment forbids the use of any built-in libraries, so we must write our own algorithm.

Algorithm: Manual Selection Sort of Top K Routes

Approach:

We don't want to sort all 265 zones (this would be $O(N \log N)$) – so we use a selection based approach that returns only the top K elements. This is more efficient if K is small (default 10) as compared to N (265).

Pseudo-code:

```
'''
```

```
function top_k_routes(zone_counts, k):
```

```
    // Input: dictionary of {zone_name: trip_count}
```

```
    // Output: list of top K zones [(name, count), ...]
```

```
    // 1. Convert dictionary to list of [name, count] pairs
```

```
    items = []
```

```
    for each (name, count) in zone_counts:
```

```
        items.append([name, count])
```

```
    n = length(items)
```

```
    k = min(k, n) // Can't return more than available
```

```
    used = array of false (size n)
```

```
    result = []
```

```
    // 2. Find top K using selection
```

```
    for i = 0 to k-1:
```

```
        best_index = -1
```

```
        best_value = -1
```

```
        for j = 0 to n-1:
```

```
            if not used[j] and items[j][1] > best_value:
```

```
                best_value = items[j][1]
```

```
                best_index = j
```

```
            if best_index == -1:
```

```
                break
```

```
            used[best_index] = true
```

```
            result.append((items[best_index][0], items[best_index][1]))
```

```
    return result
```

```
'''
```

Time Complexity: $O(K \cdot N)$

- N = number of zones (265)
- K - number of top zones to search for (default 10)
- Best case: $K = 1 \rightarrow O(N)$
- Worst case: $K = N \rightarrow O(N^2)$

Space Complexity: $O(N)$

- items array: $O(N)$
- used array: $O(N)$
- result array: $O(K)$

Why This Algorithm:

1. Efficiency : When K is relatively small compared to N (10 vs 265), it is faster than sorting all elements.
2. Simplicity: Comprehensible and accessible
3. No built in dependencies: does not use "heapq", "Counter", "sort", or "sorted"

Deterministic: Guarantees the same output, given the same input

Implementation

```
```python
```

```
def top_k_routes(zone_counts: dict, k: int) -> list:
 if not zone_counts or k <= 0:
 return []

 items = [[name, cnt] for name, cnt in zone_counts.items()]
 n = len(items)
 k = min(k, n)
 used = [False] * n
 result = []

 for _ in range(k):
 best_idx = -1
 best_val = -1
```

```

 for i in range(n):
 if not used[i] and items[i][1] > best_val:
 best_val = items[i][1]
 best_idx = i
 if best_idx == -1:
 break
 used[best_idx] = True
 result.append((items[best_idx][0], items[best_idx][1]))
 return result
'''

```

#### 4. Insights and Interpretation

**Insight 1: Manhattan has the highest taxi activity.**

How derived: SQL query that groups together trips by pickup borough.

```

'''sql
SELECT
 pz.borough,
 COUNT() AS trip_count,
 ROUND(AVG(t.fare_amount), 2) AS avg_fare
FROM trips t
LEFT JOIN zones pz ON t.pu_location_id = pz.zone_id
WHERE pz.borough IS NOT NULL
GROUP BY pz.borough
ORDER BY trip_count DESC;
'''

```

***Visualization:***



#### **Interpretation:**

43% of all taxi trips in January 2019 were taken in Manhattan. When that's the case, Manhattan is the busiest of NYC's areas in terms of taxis it demands, for both business and tourist use. The data shows that changes to ride-hailing laws, traffic and infrastructure planning are needed, particularly in Manhattan and need to be applied to the unique usage patterns of the outer boroughs.

#### **Insight 2: Peak Hour is 6 PM**

How derived: Hourly aggregation query using SQL `strftime`.

```
```sql
```

```
SELECT
```

```
    CAST(strftime('%H', pickup_datetime) AS INTEGER) AS hour,
```

```
    COUNT() AS trip_count
```

```
FROM trips
```

```
WHERE pickup_datetime IS NOT NULL
```

```
GROUP BY hour
```

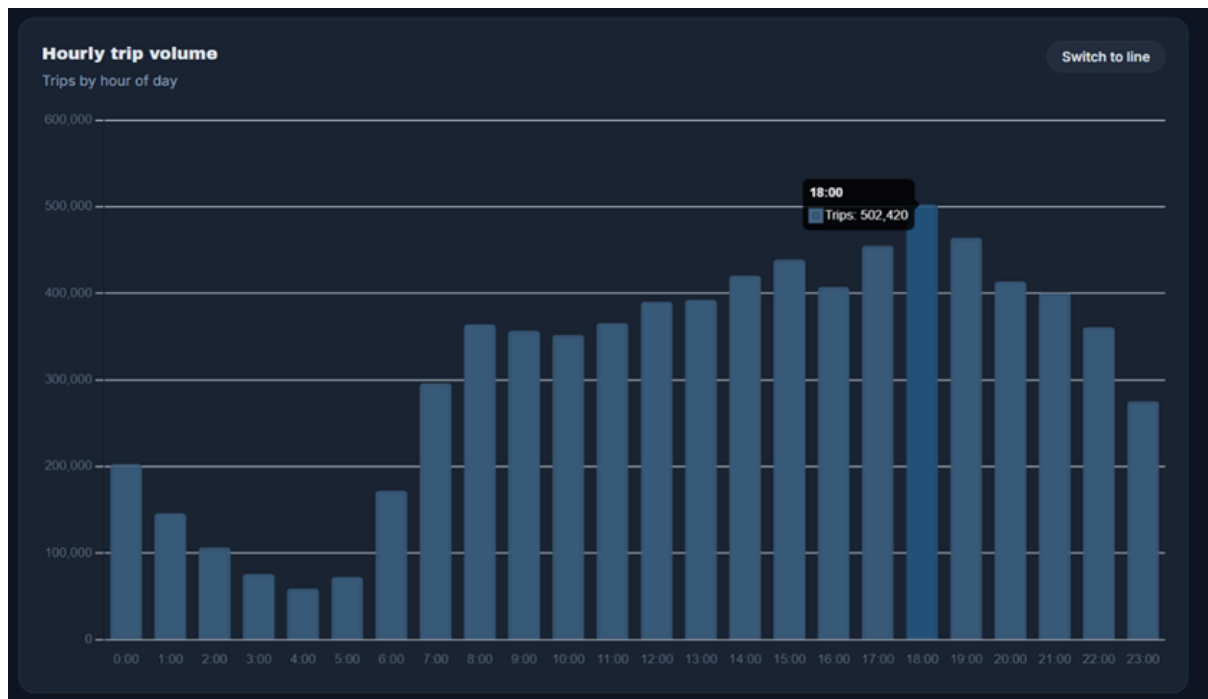
```
ORDER BY trip_count DESC
```

```
LIMIT 1;
```

```
```
```

#### **Visualization:**





### ***Interpretation:***

The peak hour is 18:00 (6 PM) with approximately 500,000 trips. This is during the "rush hour" of the evening when people are leaving work, going out to dinner or attending evening function. This can be useful for:

- Taxi dispatch optimization
- Surge pricing strategies
- Terminal Design and Layout

### **Insight 3: Card Payments Dominate**

How derived: Payment type aggregation.

```
```sql
```

```
SELECT
```

```
    payment_type,
```

```
    COUNT() AS trip_count,
```

```
    ROUND(COUNT() 100.0 / (SELECT COUNT() FROM trips), 2) AS percentage
```

```
FROM trips
```

```
WHERE payment_type IS NOT NULL
```

```
GROUP BY payment_type
```

```
ORDER BY trip_count DESC;
```

```
'''
```

Visualization:



Interpretation:

65% of all trips are paid for with a credit card and 30% by cash. This suggests that there is an inclination towards electronic payments which can be attributed to:

- Convenience for passengers
- Contactless payment adoption
- App-based ride-hailing competition
- Minimized risk of fraud to drivers

5. Reflection and Future Work

Technical Challenges

1. Data Volume: With 7.49 million records to process, memory and performance became key considerations. We tackled this by doing chunked processing (100K rows at a time) and SQLite WAL mode.
2. Map Performance: Many complex polygons (263) were slow to render. We reduced polygon styles, and applied per-borough coloring, to optimize.
3. Dashboard Speed: First load was slow because of several API calls. We implemented:
 - Browser Caching (5-minute TTL)
 - All foreign keys indexed in the database.
 - Pre-calculated summary_cache table

Team Challenges

1. Task Dependencies: Philip had to run schema.sql to get ready for Davine to insert data. To overcome this, we decided to do a schema design first.
2. Integration: API response formats needed coordination between Philip (backend) and Joseph (frontend). Early prototyping using mock data.
3. Data Consistency: It was difficult to make sure the same data was available in the different development environments. Added the schema and data files were added by relative path.

Future Improvements

1. Time-Series Analysis: Include day-over-day and week-over-week comparisons to highlight trends.
2. Predictive Modelling: Forecast trip demand and fare revenues using historical data.
3. Real-Time Data: Integrate with live TLC data feed for real-time monitoring.
4. User Authentication – enable users to save personal filters, dashboards.
5. Deployment: deploy to cloud (Render/Heroku/AWS) for public use.
6. More Data Sources: Connect with weather data, event data, and subway ridership for more comprehensive data.
7. Mobile App: Create a mobile app (React Native/Flutter).
8. Advanced Analytics: Plug in Clustering Analysis to find similar zones and anomaly detection to detect abnormal trip patterns, and route optimization.